



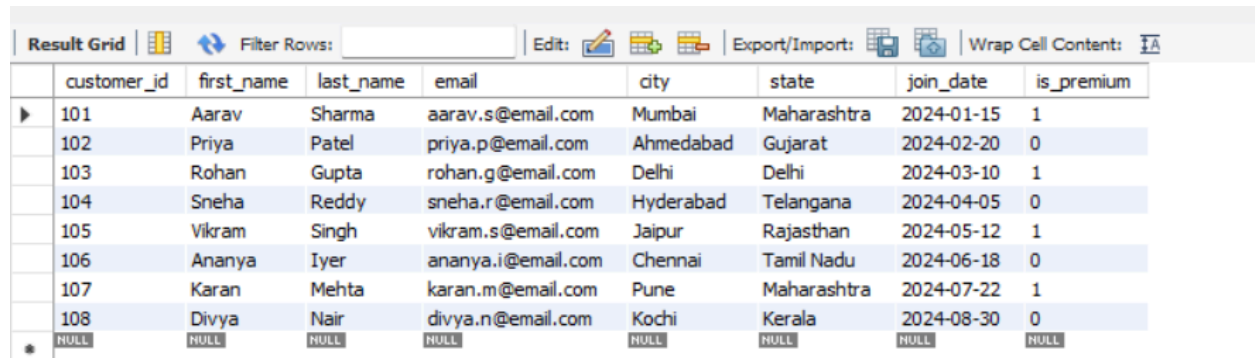
Celebal Summer Internship 2026

Week 2 Task: E-Commerce Sales Database

SECTION - A — SQL Basics (SELECT, Constraints, Primary Keys)

Q1. Write a query to display all columns and rows from the customer's table.

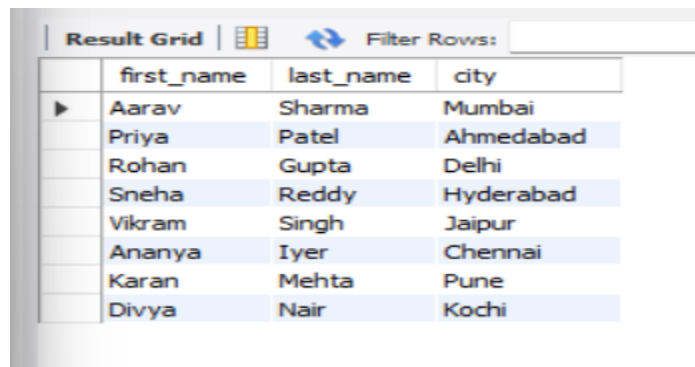
Ans: `SELECT * FROM customers;`



	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q2. Retrieve only the first_name, last_name, and city of all customers.

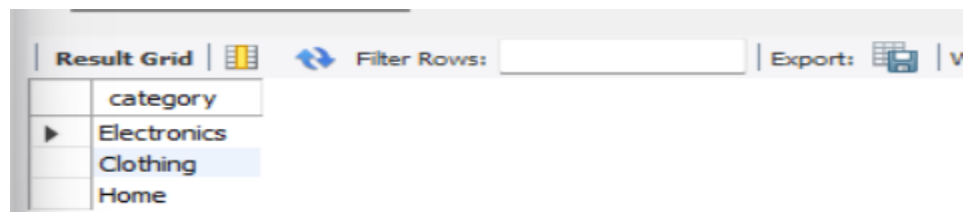
Ans: `SELECT first_name, last_name, city`
`FROM customers;`



	first_name	last_name	city
▶	Aarav	Sharma	Mumbai
	Priya	Patel	Ahmedabad
	Rohan	Gupta	Delhi
	Sneha	Reddy	Hyderabad
	Vikram	Singh	Jaipur
	Ananya	Iyer	Chennai
	Karan	Mehta	Pune
	Divya	Nair	Kochi

Q3. List all unique categories available in the products table.

Ans: `SELECT DISTINCT category`
`FROM products;`



	category
▶	Electronics
	Clothing
	Home

Q4. Identify the Primary Key of each table in the schema. Explain why a Primary Key must be unique and NOT NULL.

Ans: 1. For Customers table: The primary key of the **customers** table is **customer_id**. It uniquely identifies each customer record.

SHOW KEYS FROM customers WHERE Key_name = 'PRIMARY';

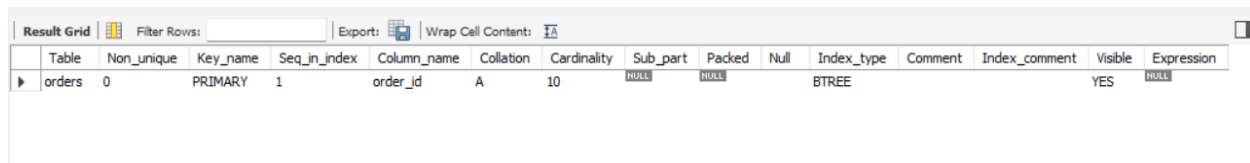


The screenshot shows the 'Result Grid' in SQL Server Enterprise Manager. The table 'customers' is selected, and the primary key 'PRIMARY' is highlighted. The key is on 'customer_id' (Seq_in_index: 1, Cardinality: 8, Index_type: BTREE). The 'Visible' column is 'YES' and the 'Expression' column is 'NULL'.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
customers	0	PRIMARY	1	customer_id	A	8	NULL	NULL		BTREE			YES	NULL

2. For orders table: The primary key of the **orders** table is **order_id**. It uniquely identifies each order.

SHOW KEYS FROM orders WHERE Key_name = 'PRIMARY';

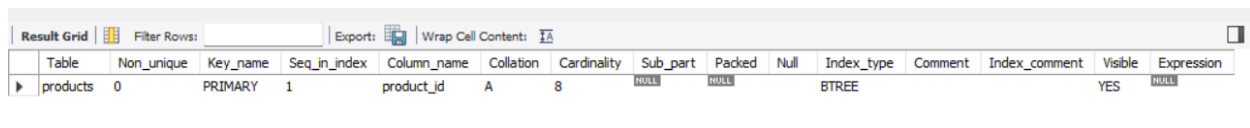


The screenshot shows the 'Result Grid' in SQL Server Enterprise Manager. The table 'orders' is selected, and the primary key 'PRIMARY' is highlighted. The key is on 'order_id' (Seq_in_index: 1, Cardinality: 10, Index_type: BTREE). The 'Visible' column is 'YES' and the 'Expression' column is 'NULL'.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	order_id	A	10	NULL	NULL		BTREE			YES	NULL

3. For products table: The primary key of the **products** table is **product_id**. It uniquely identifies each product.

SHOW KEYS FROM products WHERE Key_name = 'PRIMARY';



The screenshot shows the 'Result Grid' in SQL Server Enterprise Manager. The table 'products' is selected, and the primary key 'PRIMARY' is highlighted. The key is on 'product_id' (Seq_in_index: 1, Cardinality: 8, Index_type: BTREE). The 'Visible' column is 'YES' and the 'Expression' column is 'NULL'.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
products	0	PRIMARY	1	product_id	A	8	NULL	NULL		BTREE			YES	NULL

4. For order_items table: The primary key of the **order_items** table is **item_id**. It uniquely identifies each order item.

SHOW KEYS FROM order_items WHERE Key_name = 'PRIMARY';



The screenshot shows the 'Result Grid' in SQL Server Enterprise Manager. The table 'order_items' is selected, and the primary key 'PRIMARY' is highlighted. The key is on 'item_id' (Seq_in_index: 1, Cardinality: 15, Index_type: BTREE). The 'Visible' column is 'YES' and the 'Expression' column is 'NULL'.

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
order_items	0	PRIMARY	1	item_id	A	15	NULL	NULL		BTREE			YES	NULL

Q5. What constraints are applied to the email column in the customers table? What would happen if you tried to insert a duplicate email?

Ans: SHOW CREATE TABLE customers;

The `email` column has:

- `NOT NULL` constraint
- `UNIQUE` constraint

This ensures that every customer must have an email address and duplicate email addresses are not allowed.

Duplicate Email Test :

```
INSERT INTO customers
VALUES
(109,'Test','User','aarav.s@email.com',
'Mumbai','Maharashtra','2024-09-01',TRUE);
```

55	22:33:43	SHOW CREATE TABLE customers	1 row(s) returned	0.015 sec / 0.000 sec
56	22:33:55	INSERT INTO customers VALUES (109,'Test','User','aarav.s@email.com','Mumbai','Maharashtra','2024-09-01'...	Error Code: 1062. Duplicate entry 'aarav.s@email.com' for key 'customers.email'	0.000 sec

Error due to duplicate email because of `UNIQUE` constraint.

Q6. Try inserting a product with `unit_price = -50`. What happens and which constraint prevents it? Write both the `INSERT` statement and explain the error.

Ans:

```
INSERT INTO products
VALUES
(209,'Test Product',
'Electronics',
'TestBrand',
-50,
10);
```

The `CHECK` constraint: prevents negative prices.

`CHECK(unit_price > 0)`

Result: Constraint violation error.

57	22:36:37	INSERT INTO products VALUES (209,'Test Product','Electronics','TestBrand',-50,10)	Error Code: 3819. Check constraint 'products_chk_1' is violated.	0.000 sec
58	22:37:48	CHECK(unit_price > 0)	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to your MySQL se...	0.000 sec

Section B — Filtering & Optimization (WHERE, Indexes)

Q7. Retrieve all orders with status = 'Delivered'.

Ans: SELECT *

FROM orders

WHERE status='Delivered';

Result Grid

  Filter Rows:

Edit:   

Export/Import

	order_id	customer_id	order_date	status	total_amount
▶	1001	101	2024-08-01	Delivered	4498.00
	1002	102	2024-08-03	Delivered	799.00
	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1008	103	2024-08-20	Delivered	899.00
	1010	108	2024-08-28	Delivered	1598.00
✱	NULL	NULL	NULL	NULL	NULL

Q8. Find all products in the 'Electronics' category with a unit_price greater than ₹2000.

Ans: SELECT *

FROM products

WHERE category='Electronics'

AND unit_price > 2000;

Result Grid

Filter Rows:

Edit:

Export/Import

	product_id	product_name	category	brand	unit_price	stock_qty
▶	203	Smart Watch	Electronics	Noise	2999.00	150
	205	Bluetooth Speaker	Electronics	JBL	3499.00	200
✱	NULL	NULL	NULL	NULL	NULL	NULL

Q9. List all customers who joined in the year 2024 and belong to the state 'Maharashtra'.

Ans: SELECT *

FROM customers

WHERE state='Maharashtra'

AND join_date >= '2024-01-01'
AND join_date < '2025-01-01';

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
•	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Q10. Find all orders placed between '2024-08-10' and '2024-08-25' (inclusive) that are NOT cancelled.

Ans: SELECT *

FROM orders

WHERE order_date BETWEEN '2024-08-10'

AND '2024-08-25'

AND status <> 'Cancelled';

Result Grid

Filter Rows:

Edit:

	order_id	customer_id	order_date	status	total_amount
▶	1004	101	2024-08-10	Delivered	3499.00
	1006	105	2024-08-15	Delivered	5898.00
	1007	106	2024-08-18	Pending	1299.00
	1008	103	2024-08-20	Delivered	899.00
	1009	107	2024-08-25	Shipped	6098.00
✱	NULL	NULL	NULL	NULL	NULL

Q11. Explain what the index idx_orders_date does. How would it improve the performance of a query that filters orders by order_date? Write a sample query that would benefit from this index.

Ans: **idx_orders_date** is an index on **order_date**.

Benefits:

- Faster searching
- Faster filtering
- Faster sorting

The database engine can directly locate matching rows instead of scanning the entire table.

```
SELECT *
FROM orders
WHERE order_date='2024-08-15';
```

Result Grid

Filter Rows:

Edit:

	order_id	customer_id	order_date	status	total_amount
▶	1006	105	2024-08-15	Delivered	5898.00
✱	NULL	NULL	NULL	NULL	NULL

Q12. If you run: `SELECT * FROM customers WHERE YEAR(join_date) = 2024;` — would the index on `join_date` be used? Explain why or why not, and rewrite the query to be index-friendly (SARGable).

Ans: Non-SARGable Query:

```
SELECT *
FROM customers
WHERE YEAR(join_date)=2024;
```

Result Grid

Filter Rows:

Edit:

Export/Import:

Wrap Cell Content:

	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
✱	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Why? Applying a function on the column prevents efficient index usage.

SARGable Query:

```
SELECT *
FROM customers
WHERE join_date >= '2024-01-01'
AND join_date < '2025-01-01';
```

Result Grid   Filter Rows: Edit:    Export/Import:   Wrap Cell Content: 								
	customer_id	first_name	last_name	email	city	state	join_date	is_premium
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Section C — Aggregation (GROUP BY, SUM, COUNT, AVG, MIN, MAX)

Q13. Count the total number of orders in the orders table.

Ans: SELECT COUNT(*) AS total_orders
FROM orders;

Result Grid  	
	total_orders
▶	10

Q14. Find the total revenue (SUM of total_amount) from all 'Delivered' orders.

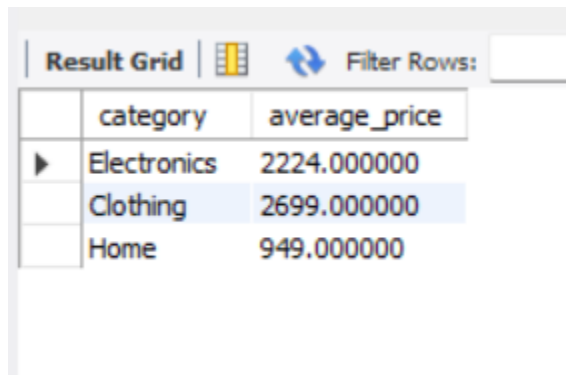
Ans: SELECT SUM(total_amount) AS total_revenue
FROM orders
WHERE status = 'Delivered';

Result Grid  	
	total_revenue
▶	17191.00

Q15. Calculate the average unit_price of products in each category.

Ans: SELECT category,
AVG(unit_price) AS average_price
FROM products

GROUP BY category;

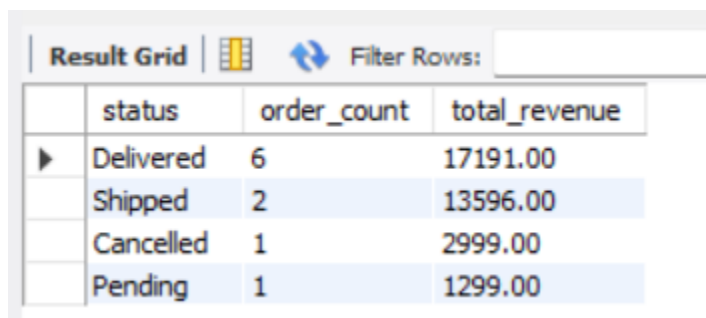


The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with two columns: 'category' and 'average_price'. The data is as follows:

category	average_price
Electronics	2224.000000
Clothing	2699.000000
Home	949.000000

Q16. For each order status, find the count of orders and the total revenue. Sort the result by total revenue in descending order.

Ans: SELECT status,
COUNT(*) AS order_count,
SUM(total_amount) AS total_revenue
FROM orders
GROUP BY status
ORDER BY total_revenue DESC;



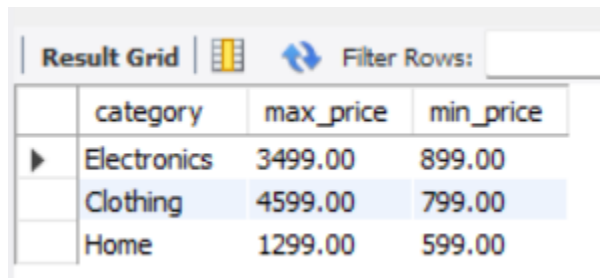
The screenshot shows a database interface with a 'Result Grid' tab. It contains a table with four columns: 'status', 'order_count', and 'total_revenue'. The data is as follows:

status	order_count	total_revenue
Delivered	6	17191.00
Shipped	2	13596.00
Cancelled	1	2999.00
Pending	1	1299.00

Q17. Find the most expensive (MAX) and cheapest (MIN) product in each category.

Ans: SELECT category,
MAX(unit_price) AS max_price,
MIN(unit_price) AS min_price
FROM products

GROUP BY category;

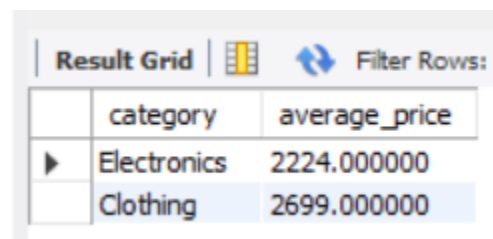


The screenshot shows a database interface with a 'Result Grid' tab. The grid displays three rows of data grouped by category. The columns are 'category', 'max_price', and 'min_price'. The rows are: Electronics (max_price: 3499.00, min_price: 899.00), Clothing (max_price: 4599.00, min_price: 799.00), and Home (max_price: 1299.00, min_price: 599.00).

	category	max_price	min_price
▶	Electronics	3499.00	899.00
	Clothing	4599.00	799.00
	Home	1299.00	599.00

Q18. List all product categories where the average unit_price is greater than ₹2000. (Hint: Use HAVING clause)

Ans: SELECT category,
AVG(unit_price) AS average_price
FROM products
GROUP BY category
HAVING AVG(unit_price) > 2000;



The screenshot shows a database interface with a 'Result Grid' tab. The grid displays two rows of data grouped by category. The columns are 'category' and 'average_price'. The rows are: Electronics (average_price: 2224.000000) and Clothing (average_price: 2699.000000).

	category	average_price
▶	Electronics	2224.000000
	Clothing	2699.000000

Section D — Joins & Relationships

Q19. Write an INNER JOIN query to display each order along with the customer's first_name and last_name. Show: order_id, order_date, first_name, last_name, total_amount.

Ans: SELECT
o.order_id,
o.order_date,
c.first_name,

```

c.last_name,
o.total_amount
FROM orders o
INNER JOIN customers c
ON o.customer_id = c.customer_id;

```

Result Grid					
Filter Rows:					
Export:  Write					
	order_id	order_date	first_name	last_name	total_amount
▶	1001	2024-08-01	Aarav	Sharma	4498.00
	1004	2024-08-10	Aarav	Sharma	3499.00
	1002	2024-08-03	Priya	Patel	799.00
	1003	2024-08-05	Rohan	Gupta	7498.00
	1008	2024-08-20	Rohan	Gupta	899.00
	1005	2024-08-12	Sneha	Reddy	2999.00
	1006	2024-08-15	Vikram	Singh	5898.00
	1007	2024-08-18	Ananya	Iyer	1299.00
	1009	2024-08-25	Karan	Mehta	6098.00
	1010	2024-08-28	Divya	Nair	1598.00

Q20. Using a LEFT JOIN, list ALL customers and their orders (if any). Customers with no orders should still appear with NULL values for order columns.

```

Ans: SELECT
c.customer_id,
c.first_name,
c.last_name,
o.order_id,
o.order_date
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id;

```

Result Grid					
Filter Rows:					
Export:					
	customer_id	first_name	last_name	order_id	order_date
▶	101	Aarav	Sharma	1001	2024-08-01
	101	Aarav	Sharma	1004	2024-08-10
	102	Priya	Patel	1002	2024-08-03
	103	Rohan	Gupta	1003	2024-08-05
	103	Rohan	Gupta	1008	2024-08-20
	104	Sneha	Reddy	1005	2024-08-12
	105	Vikram	Singh	1006	2024-08-15
	106	Ananya	Iyer	1007	2024-08-18
	107	Karan	Mehta	1009	2024-08-25
	108	Divya	Nair	1010	2024-08-28

Q21. Write a query using JOINS across three tables (orders → order_items → products) to show: order_id, product_name, quantity, unit_price, and discount_pct for each order item.

Ans: SELECT

o.order_id,

p.product_name,

oi.quantity,

oi.unit_price,

oi.discount_pct

FROM orders o

JOIN order_items oi

ON o.order_id = oi.order_id

JOIN products p

ON oi.product_id = p.product_id;

Result Grid

Filter Rows:

Export:

Wrap Cell Contents

	order_id	product_name	quantity	unit_price	discount_pct
▶	1001	Wireless Earbuds	2	1499.00	0.00
	1006	Wireless Earbuds	1	1499.00	10.00
	1002	Cotton T-Shirt	1	799.00	0.00
	1003	Smart Watch	1	2999.00	0.00
	1005	Smart Watch	1	2999.00	0.00
	1003	Running Shoes	1	4599.00	5.00
	1006	Running Shoes	1	4599.00	5.00
	1004	Bluetooth Speaker	1	3499.00	0.00
	1009	Bluetooth Speaker	1	3499.00	0.00
	1007	Bedsheet Set	1	1299.00	0.00
	1010	Bedsheet Set	1	1299.00	0.00
	1001	Laptop Stand	1	899.00	10.00

Q22. Explain the difference between LEFT JOIN and RIGHT JOIN with an example from this schema. When would you use a FULL OUTER JOIN?

Ans: LEFT JOIN :

- Returns all rows from the left table.
- Matching rows from the right table are returned.
- Non-matching rows show NULL values.

SELECT *

FROM customers c

LEFT JOIN orders o

ON c.customer_id = o.customer_id;

Result Grid		Filter Rows:		Export:		Wrap Cell Content:							
	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00

RIGHT JOIN :

- Returns all rows from the right table.
- Matching rows from the left table are returned.
- Non-matching rows show NULL values.

SELECT *

FROM customers c

RIGHT JOIN orders o

ON c.customer_id = o.customer_id;

Result Grid													
Filter Rows:				Export:		Wrap Cell Content:							
	customer_id	first_name	last_name	email	city	state	join_date	is_premium	order_id	customer_id	order_date	status	total_amount
▶	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1001	101	2024-08-01	Delivered	4498.00
	102	Priya	Patel	priya.p@email.com	Ahmedabad	Gujarat	2024-02-20	0	1002	102	2024-08-03	Delivered	799.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1003	103	2024-08-05	Shipped	7498.00
	101	Aarav	Sharma	aarav.s@email.com	Mumbai	Maharashtra	2024-01-15	1	1004	101	2024-08-10	Delivered	3499.00
	104	Sneha	Reddy	sneha.r@email.com	Hyderabad	Telangana	2024-04-05	0	1005	104	2024-08-12	Cancelled	2999.00
	105	Vikram	Singh	vikram.s@email.com	Jaipur	Rajasthan	2024-05-12	1	1006	105	2024-08-15	Delivered	5898.00
	106	Ananya	Iyer	ananya.i@email.com	Chennai	Tamil Nadu	2024-06-18	0	1007	106	2024-08-18	Pending	1299.00
	103	Rohan	Gupta	rohan.g@email.com	Delhi	Delhi	2024-03-10	1	1008	103	2024-08-20	Delivered	899.00
	107	Karan	Mehta	karan.m@email.com	Pune	Maharashtra	2024-07-22	1	1009	107	2024-08-25	Shipped	6098.00
	108	Divya	Nair	divya.n@email.com	Kochi	Kerala	2024-08-30	0	1010	108	2024-08-28	Delivered	1598.00

FULL OUTER JOIN

- Returns all matching and non-matching rows from both tables.
- Useful when complete data from both tables is required.

Q23. Identify all Foreign Key relationships in the schema. Explain what would happen if you tried to insert an order with customer_id = 999 (which doesn't exist in customers).

Ans: SELECT

TABLE_NAME,

COLUMN_NAME,

REFERENCED_TABLE_NAME,

REFERENCED_COLUMN_NAME

FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE

WHERE REFERENCED_TABLE_NAME IS NOT NULL

AND TABLE_SCHEMA = 'celebal_week2';

Result Grid Filter Rows: Export: Wrap Cell Content: ☐				
	TABLE_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
▶	order_items	order_id	orders	order_id
	order_items	product_id	products	product_id
	orders	customer_id	customers	customer_id

Foreign Keys:

- orders.customer_id → customers.customer_id
- order_items.order_id → orders.order_id
- order_items.product_id → products.product_id

Invalid Insert Example : Foreign Key Constraint Error because customer_id 999 does not exist.

✓	78	23:08:54	SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME, REFERENCED_COLUMN_NAME...	3 row(s) returned	0.016 sec / 0.000 sec
✗	79	23:13:37	INSERT INTO orders VALUES (2000,999,2024-09-01,'Pending',1000)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('celebal_week2', 'orders', 'CO...)	0.015 sec

Section E — Advanced Concepts (CASE, ACID, Transactions)

Q24. Write a query using CASE to classify products into price tiers:

- 'Budget' → unit_price < 1000
- 'Mid-Range' → unit_price BETWEEN 1000 AND 3000
- 'Premium' → unit_price > 3000

Display: product_name, unit_price, price_tier.

Ans: SELECT

product_name,

unit_price,

CASE

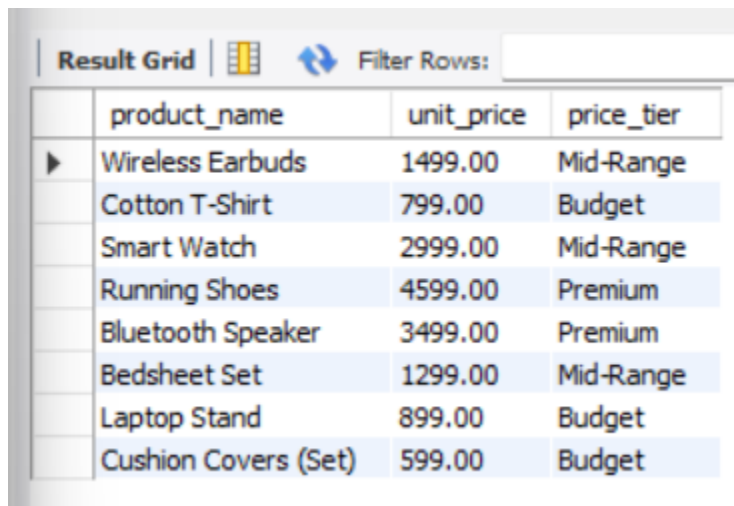
WHEN unit_price < 1000 THEN 'Budget'

WHEN unit_price BETWEEN 1000 AND 3000 THEN 'Mid-Range'

ELSE 'Premium'

END AS price_tier

FROM products;



A screenshot of a database application's 'Result Grid'. The grid has a header row with columns: product_name, unit_price, and price_tier. Below the header, there are eight data rows, each representing a different product. The first row is 'Wireless Earbuds' with a unit price of 1499.00 and a price tier of 'Mid-Range'. The second row is 'Cotton T-Shirt' with a unit price of 799.00 and a price tier of 'Budget'. The third row is 'Smart Watch' with a unit price of 2999.00 and a price tier of 'Mid-Range'. The fourth row is 'Running Shoes' with a unit price of 4599.00 and a price tier of 'Premium'. The fifth row is 'Bluetooth Speaker' with a unit price of 3499.00 and a price tier of 'Premium'. The sixth row is 'Bedsheet Set' with a unit price of 1299.00 and a price tier of 'Mid-Range'. The seventh row is 'Laptop Stand' with a unit price of 899.00 and a price tier of 'Budget'. The eighth row is 'Cushion Covers (Set)' with a unit price of 599.00 and a price tier of 'Budget'. The grid is titled 'Result Grid' and has a 'Filter Rows:' button.

	product_name	unit_price	price_tier
▶	Wireless Earbuds	1499.00	Mid-Range
	Cotton T-Shirt	799.00	Budget
	Smart Watch	2999.00	Mid-Range
	Running Shoes	4599.00	Premium
	Bluetooth Speaker	3499.00	Premium
	Bedsheet Set	1299.00	Mid-Range
	Laptop Stand	899.00	Budget
	Cushion Covers (Set)	599.00	Budget

Q25. Using a CASE statement inside an aggregate function, count how many orders are 'Delivered' vs 'Not Delivered' (all other statuses). Display the result in a single row.

Ans: SELECT

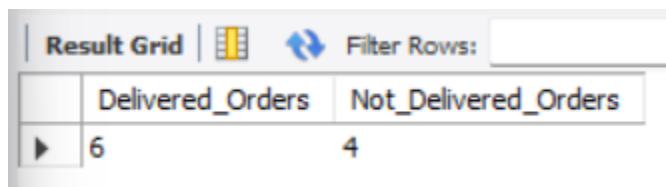
SUM(CASE WHEN status='Delivered' THEN 1 ELSE 0 END)

AS Delivered_Orders,

SUM(CASE WHEN status<>'Delivered' THEN 1 ELSE 0 END)

AS Not_Delivered_Orders

FROM orders;



A screenshot of a database application's 'Result Grid'. The grid has a header row with columns: Delivered_Orders and Not_Delivered_Orders. Below the header, there is one data row showing the counts: 6 for Delivered_Orders and 4 for Not_Delivered_Orders. The grid is titled 'Result Grid' and has a 'Filter Rows:' button.

	Delivered_Orders	Not_Delivered_Orders
▶	6	4

Q26. Explain each letter of ACID:

- A – Atomicity
- C – Consistency
- I – Isolation

- D – Durability

Give a real-world example (e.g., bank transfer) showing why each property is important.

Ans: A - Atomicity

- Either all operations succeed or none do.
- Example: Bank transfer should debit and credit together.

START TRANSACTION;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 201;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 999;

ROLLBACK;

✓	89	23:31:12	START TRANSACTION	0 row(s) affected	0.000 sec
✓	90	23:31:12	UPDATE products SET stock_qty = stock_qty - 1...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
✓	91	23:31:12	UPDATE products SET stock_qty = stock_qty - 1...	0 row(s) affected Rows matched: 0 Changed: 0 ...	0.000 sec
✓	92	23:31:12	ROLLBACK	0 row(s) affected	0.000 sec

C - Consistency

- Data remains valid before and after a transaction.

INSERT INTO products

VALUES

(300,'Test Product',

'Electronics',

'Brand',

-100,
10);

✖ 93 23:32:16 INSERT INTO products VALUES (300,'Test Prod... Error Code: 3819. Check constraint 'products_chk... 0.000 sec

I - Isolation

- Multiple transactions do not interfere with each other.

Transaction 1: START TRANSACTION;

UPDATE products

SET stock_qty = stock_qty - 5

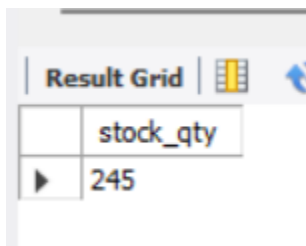
WHERE product_id = 201;

✔ 94 23:33:06 START TRANSACTION 0 row(s) affected 0.016 sec
✔ 95 23:33:06 UPDATE products SET stock_qty = stock_qty - 5... 1 row(s) affected Rows matched: 1 Changed: 1 ... 0.000 sec

Transaction 2: SELECT stock_qty

FROM products

WHERE product_id = 201;



	stock_qty
▶	245

D - Durability

- Once committed, data remains saved permanently.

START TRANSACTION;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 201;

COMMIT;

✓	97	23:35:00	START TRANSACTION	0 row(s) affected	0.031 sec
✓	98	23:35:00	UPDATE products SET stock_qty = stock_qty - 1...	1 row(s) affected Rows matched: 1 Changed: 1 ...	0.000 sec
✓	99	23:35:00	COMMIT	0 row(s) affected	0.000 sec

Verification Query

SELECT stock_qty

FROM products

WHERE product_id = 201;

Result Grid	Filter
stock_qty	
▶ 244	

Q27. Write a SQL transaction that does the following atomically:

1. Insert a new order (order_id=1011, customer_id=102, today's date, 'Pending', 1598.00)
 2. Insert two order items for that order
 3. Update the stock_qty of the purchased products
 4. If any step fails, ROLLBACK the entire transaction. Otherwise, COMMIT.
- Write the complete BEGIN...COMMIT/ROLLBACK block.

Ans:

START TRANSACTION;

INSERT INTO orders

VALUES

(1011,102,CURDATE(),'Pending',1598.00);

INSERT INTO order_items

VALUES

(5016,1011,206,1,1299.00,0);

INSERT INTO order_items

VALUES

(5017,1011,208,1,299.00,0);

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 206;

UPDATE products

SET stock_qty = stock_qty - 1

WHERE product_id = 208;

COMMIT;

✓	82	23:24:50	START TRANSACTION	0 row(s) affected	0.000 sec
✓	83	23:24:50	INSERT INTO orders VALUES (1011,102,CURDATE(), 'Pending',1598.00)	1 row(s) affected	0.000 sec
✓	84	23:24:50	INSERT INTO order_items VALUES (5016,1011,206,1,1299.00,0)	1 row(s) affected	0.000 sec
✓	85	23:24:50	INSERT INTO order_items VALUES (5017,1011,208,1,299.00,0)	1 row(s) affected	0.000 sec
✓	86	23:24:50	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 206	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
✓	87	23:24:50	UPDATE products SET stock_qty = stock_qty - 1 WHERE product_id = 208	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
✓	88	23:24:50	COMMIT	0 row(s) affected	0.000 sec